

Programación dinámica

TDA / Algo3

Departamento de Computación
Facultad de Ciencias Exactas y Naturales
Universidad de Buenos Aires

2do Cuatrimestre de 2026

Plan

- 1 Nueva técnica: Programación Dinámica
 - Ejemplo: Problema de la Mochila.
 - Análisis típico de la complejidad de un algoritmo con PD.
- 2 Corte de varillas: un caso de ventaja exponencial.
- 3 Producto de matrices.
- 4 Conjunto independiente: un caso negativo D:
- 5 Resumen general.
- 6 Reconstrucción de la solución.

Programación Dinámica

La programación dinámica (PD) puede pensarse como un **tipo de poda** que se aplica sobre un algoritmo recursivo (tipo un backtracking).

Idea

Guardar el valor asociado a cada nodo del árbol de backtracking para evitar **recomputarlos**.

Vamos a ver muchos ejemplos para ilustrar la idea.

Problema de la mochila

Definición

Tenemos n objetos. El objeto i tiene peso entero $a_i > 0$ y valor $b_i \geq 0$. La capacidad W es un entero no negativo. Buscamos

$$\max_{S \subseteq \{1, \dots, n\}} \sum_{i \in S} b_i \quad \text{sujeto a} \quad \sum_{i \in S} a_i \leq W.$$

Objeto i	1	2	3
Peso a_i	3	2	2
Valor b_i	5	3	4

Capacidad $W = 5$.

Elegir $S = \{1, 3\}$ ocupa peso 5 y obtiene valor 9.

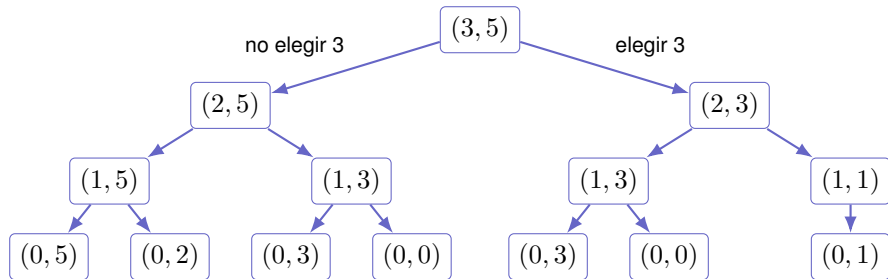
El problema de la mochila: con backtracking

Ya sabemos resolver este problema con una función recursiva.

$$F(i, w) = \begin{cases} -\infty, & w < 0, \\ 0, & i = 0, \\ \max\{F(i-1, w), b_i + F(i-1, w - a_i)\}, & i > 0. \end{cases}$$

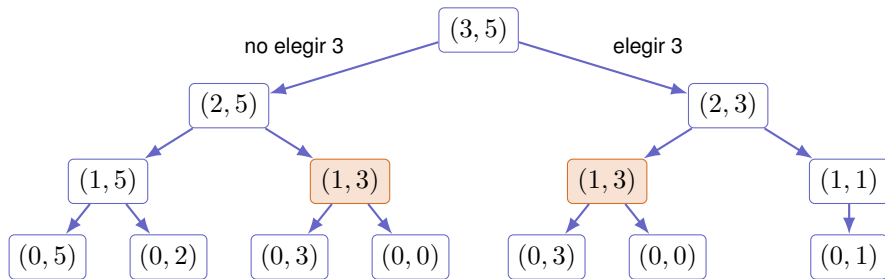
La respuesta a la instancia original es $F(n, W)$. El algoritmo tiene complejidad de peor caso $O(2^n)$.

El problema de la mochila: el árbol de backtracking.



El problema de la mochila: el árbol de backtracking.

Podemos asociar a cada nodo del árbol los parámetros de su llamado recursivo. Luego, hay casos en los que varios nodos están asociados a llamados idénticos.



Ambos nodos hacen **exactamente** los mismos cálculos. La idea entonces es guardar el resultado cuando se procesa uno de estos nodos para ya tener el valor listo cuando se procesa el segundo.

El problema de la mochila: memorizar las soluciones

Cada nodo se asocia a un conjunto de parámetros de la función, y la función toma dos naturales. Luego, podemos usar una matriz para guardar los valores.

Estructura de memorización

Usamos una matriz $M[0..n, 0..W]$. La idea es que $M[i, w]$ guarde $F(i, w)$ una vez que se lo calcule.

- Con esta estructura consultar o escribir una celda cuesta $O(1)$.
- Inicialmente, todas las celdas están en un valor nulo $\perp$.

El problema de la mochila: backtracking normal

MOCHILA (i, w)

si $w < 0$

devolver $-\infty$

si $i = 0$

devolver 0

devolver $\max(\text{MOCHILA}(i-1, w), \text{MOCHILA}(i-1, w - a_i) + b_i)$

El problema de la mochila: backtracking con memoria

MOCHILA-PD(i, w)

si $w < 0$

devolver $-\infty$

si $i = 0$

devolver 0

si $M[i, w] \neq \perp$

devolver $M[i, w]$

$M[i, w] \leftarrow \max(\text{MOCHILA-PD}(i-1, w), \text{MOCHILA-PD}(i-1, w - a_i) + b_i)$

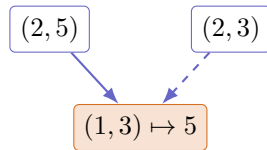
devolver $M[i, w]$

Antes de ejecutar $\text{MOCHILA-PD}(n, W)$ hay que inicializar la memoria con $\perp$ en todas las posiciones. Notar qué parecido a una poda es el cacho de código que agregamos.

Ejemplo de memorización

En el árbol de ejemplo que vimos antes, la memoria se va a usar para no recalcular el nodo con valor $(1, 3)$

- 1 Desde $(2, 5)$ se calcula $F(1, 3) = 5$.
- 2 Antes de volver, se escribe $M[1, 3] = 5$.
- 3 Desde $(2, 3)$ se vuelve a pedir $F(1, 3)$.
- 4 La consulta devuelve 5 sin hacer recursión.



Mismo subproblema!

Cuándo ayuda la memoria?

En el fondo, ahora la complejidad va a depender de la *cantidad de estados distintos*, ya que estados con misma etiqueta se procesan una única vez.

Pensemos mejor esto último.

Cada estado es visitado a lo sumo dos veces

Un estado (i, w) , con $i < n$, solo puede ser llamado desde:

$$\underbrace{(i+1, w)}_{\text{no elegir } i+1} \quad \text{o} \quad \underbrace{(i+1, w + a_{i+1})}_{\text{elegir } i+1}.$$

Teniendo en cuenta que cada nodo se expande una única vez, concluimos que cada nodo es procesado a lo sumo dos veces.

El costo total incluye visitas e inicialización

- Ya sabemos que cada nodo se visita una única vez. Ahora... Cuántos nodos distintos hay?

El costo total incluye visitas e inicialización

- Ya sabemos que cada nodo se visita una única vez. Ahora... Cuántos nodos distintos hay? Cómo hay nW etiquetas distintas, a lo sumo se van a visitar nW nodos distintos.
- Cada visita hace $O(1)$ operaciones. Luego, procesar todos los nodos toma $O(nW)$.

El costo total incluye visitas e inicialización

- Ya sabemos que cada nodo se visita una única vez. Ahora... Cuántos nodos distintos hay? Cómo hay nW etiquetas distintas, a lo sumo se van a visitar nW nodos distintos.
- Cada visita hace $O(1)$ operaciones. Luego, procesar todos los nodos toma $O(nW)$.
- La matriz de memoria tiene $O(nW)$ posiciones, y llenarla al principio toma $O(nW)$.

La complejidad final entonces es...

El costo total incluye visitas e inicialización

- Ya sabemos que cada nodo se visita una única vez. Ahora... Cuántos nodos distintos hay? Cómo hay nW etiquetas distintas, a lo sumo se van a visitar nW nodos distintos.
- Cada visita hace $O(1)$ operaciones. Luego, procesar todos los nodos toma $O(nW)$.
- La matriz de memoria tiene $O(nW)$ posiciones, y llenarla al principio toma $O(nW)$.

La complejidad final entonces es... $O(nW)$

Complejidad final

Algoritmo	Tiempo	Espacio auxiliar
Backtracking	$\mathcal{O}(2^n)$	$\mathcal{O}(n)$
PD	$\Theta(nW)$	$\Theta(nW)$

Cuál es mejor?

Complejidad final

Algoritmo	Tiempo	Espacio auxiliar
Backtracking	$\mathcal{O}(2^n)$	$\mathcal{O}(n)$
PD	$\Theta(nW)$	$\Theta(nW)$

Cuál es mejor?

Comparación de las cotas

En principio son incomparables, ya que la primera **no depende** de W .

No obstante, si nos dan garantías sobre W (por ejemplo, que es $\mathcal{O}(n^4)$) entonces el algoritmo de PD es mejor. En general PD va a convenir si W es más chico que $2^n/n$.

Complejidad final

- Si W es muy grande el algoritmo de PD puede ser muy malo.
- Peor aún, si $W = 2^n$, llenar la tabla cuesta $\Theta(n2^n)$, aun antes de empezar la recursión. Esto es muchísima memoria

Complejidad final

- Si W es muy grande el algoritmo de PD puede ser muy malo.
- Peor aún, si $W = 2^n$, llenar la tabla cuesta $\Theta(n2^n)$, aun antes de empezar la recursión. Esto es muchísima memoria
- Si W es muy grande es probable que no todas las celdas de memoria se usen. Incluso podría ser preferible usar otra estructura de memoria en ese caso.

En general, cambiar la estructura de memoria nos va a impactar en (1) el costo de la inicialización, y (2) el costo por acceso. Por ejemplo, si usamos un ABB balanceado tendríamos inicialización $O(1)$ pero el costo por consulta pasa a $O(\log NW)$ (en peor caso).

Cortar varillas

Tenemos una varilla de longitud entera $n \geq 1$ y precios $p_1, \dots, p_n$: una pieza de longitud k se vende por p_k .

Objetivo

Queremos dividir la varilla en piezas de longitudes enteras positivas que sumen n , de modo de maximizar el ingreso total. Se permite venderla sin cortar, y los cortes en si no tienen costo.

longitud k	1	2	3	4
precio p_k	1	5	8	9

Una varilla de longitud 4 se podría vender por 9 pesos o bien se la podría partir en dos partes y vender a 5 cada una.

Cortar varillas

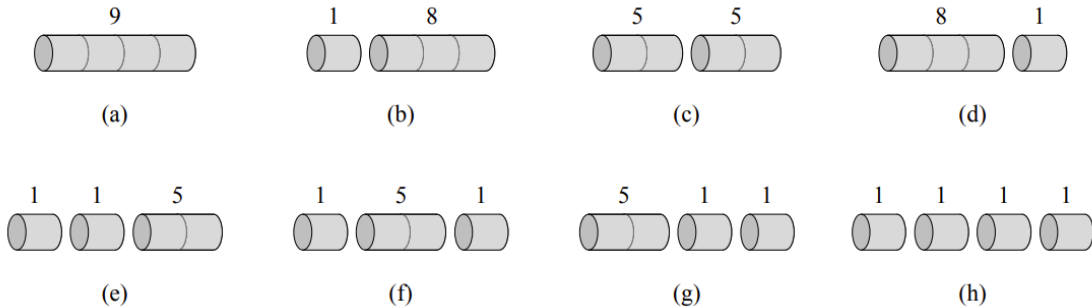


Figura: Tomado del Cormen

Cortar varillas

Llamemos $R(\ell)$ al mayor ingreso que podemos obtener de una varilla de longitud ℓ . Cuál es una posible definición recursiva para R ?

Cortar varillas

Llamemos $R(\ell)$ al mayor ingreso que podemos obtener de una varilla de longitud ℓ . Cuál es una posible definición recursiva para R ?

Un algoritmo de backtracking?

$$R(\ell) = \begin{cases} 0, & \ell = 0, \\ \max_{1 \leq k \leq \ell} \{p_k + R(\ell - k)\}, & \ell > 0. \end{cases}$$

- Vendemos una primera pieza de longitud k por p_k .
- Resolvemos recursivamente el resto, de longitud $\ell - k$.

La llamada inicial es $R(n)$.

Cortar varillas

CORTAR(ℓ)

si $\ell = 0$: **devolver** 0

$q \leftarrow -\infty$

para $k = 1, \dots, \ell$

$r \leftarrow \text{CORTAR}(\ell - k)$

$q \leftarrow \max\{q, p_k + r\}$

devolver q

Complejidad del backtracking

Para ver la complejidad, calculemos la cantidad de llamados y su costo. Empecemos con lo fácil:
Cuánto es el costo por llamado?

Complejidad del backtracking

Para ver la complejidad, calculemos la cantidad de llamados y su costo. Empecemos con lo fácil: Cuánto es el costo por llamado? Peor caso, $O(\ell)$. Ahora calculemos la cantidad de nodos del árbol.

Complejidad del backtracking

Para ver la complejidad, calculemos la cantidad de llamados y su costo. Empecemos con lo fácil: Cuánto es el costo por llamado? Peor caso, $O(\ell)$. Ahora calculemos la cantidad de nodos del árbol. Sea $C(\ell)$ la cantidad total de llamadas. Luego:

$$C(0) = 1, \quad C(\ell) = 1 + \sum_{j=0}^{\ell-1} C(j).$$

Para $\ell \geq 1$, al separar el último sumando obtenemos

$$C(\ell) = C(\ell - 1) + \underbrace{\left(1 + \sum_{j=0}^{\ell-2} C(j) \right)}_{C(\ell-1)} = 2C(\ell - 1).$$

Luego $C(\ell) = 2^\ell$. Concluimos entonces que la complejidad del backtracking es

Complejidad del backtracking

Para ver la complejidad, calculemos la cantidad de llamados y su costo. Empecemos con lo fácil: Cuánto es el costo por llamado? Peor caso, $O(\ell)$. Ahora calculemos la cantidad de nodos del árbol. Sea $C(\ell)$ la cantidad total de llamadas. Luego:

$$C(0) = 1, \quad C(\ell) = 1 + \sum_{j=0}^{\ell-1} C(j).$$

Para $\ell \geq 1$, al separar el último sumando obtenemos

$$C(\ell) = C(\ell - 1) + \underbrace{\left(1 + \sum_{j=0}^{\ell-2} C(j) \right)}_{C(\ell-1)} = 2C(\ell - 1).$$

Luego $C(\ell) = 2^\ell$. Concluimos entonces que la complejidad del backtracking es $O(n2^n)$.

Memorizando las soluciones

Podemos usar un vector $M[0..n]$:

$$M[\ell] = \begin{cases} \perp, & \text{si todavía no calculamos } R(\ell), \\ R(\ell), & \text{si ya lo calculamos.} \end{cases}$$

Cuánto cuesta inicializarlo?

Memorizando las soluciones

Podemos usar un vector $M[0..n]$:

$$M[\ell] = \begin{cases} \perp, & \text{si todavía no calculamos } R(\ell), \\ R(\ell), & \text{si ya lo calculamos.} \end{cases}$$

Cuánto cuesta inicializarlo? $O(n)$. Y leerlo / modificarlo?

Memorizando las soluciones

Podemos usar un vector $M[0..n]$:

$$M[\ell] = \begin{cases} \perp, & \text{si todavía no calculamos } R(\ell), \\ R(\ell), & \text{si ya lo calculamos.} \end{cases}$$

Cuánto cuesta inicializarlo? $O(n)$. Y leerlo / modificarlo? $O(1)$.

Varillas, ahora con PD

CORTAR-MEM(ℓ)

si $\ell = 0$:

devolver 0

si $M[\ell] \neq \perp$:

devolver $M[\ell]$

$q \leftarrow -\infty$

para $k = 1, \dots, \ell$

$r \leftarrow \text{CORTAR-MEM}(\ell - k)$

$q \leftarrow \max\{q, p_k + r\}$

$M[\ell] \leftarrow q$

devolver q

Análisis de PD

- Como antes, al usar PD garantizamos que cada nodo hace sus llamados recursivo una única vez. Teniendo esto en cuenta: cuántas veces se visita cada estado?

Análisis de PD

- Como antes, al usar PD garantizamos que cada nodo hace sus llamados recursivo una única vez. Teniendo esto en cuenta: cuántas veces se visita cada estado?
- Para contar, podemos imaginar que cada padre consulta la memoria **antes** de llamar: así sólo se llama recursivamente a estados no memorizados.
- No lo hacemos así en la implementación porque queda estéticamente horrible (aunque en términos de performance es muy parecido).

Complejidad de PD

Por lo tanto, la memoria se inicializa en $O(n)$, y luego tenemos n estados distintos, donde cada uno se procesa en $O(n)$ y se visita una única vez. Cuál es la complejidad final?

Complejidad de PD

Por lo tanto, la memoria se inicializa en $O(n)$, y luego tenemos n estados distintos, donde cada uno se procesa en $O(n)$ y se visita una única vez. Cuál es la complejidad final? $O(n^2)$

Algoritmo	Tiempo	Espacio adicional
Recursivo sin memoria	$\Theta(n2^n)$	$\Theta(n)$
Recursivo con memoria	$\Theta(n^2)$	$\Theta(n)$

Programación dinámica gana

En este caso la complejidad de PD es estrictamente mejor. Agregar memorización mejoró exponencialmente el algoritmo.

Podemos pensarlo así: en los 2^n nodos había un montón que representaban exactamente la misma instancia. A esta propiedad se la llama **superposición de subproblemas**.

Multiplicación de matrices

Queremos calcular un producto de matrices:

$$A_1 A_2 \cdots A_n,$$

donde cada A_i tiene tamaño $d_{i-1} \times d_i$.

Objetivo

Elegir una parentización que minimice el número de multiplicaciones escalares.

- Multiplicar una matriz de dimensiones $a \times b$ por otra con dimensiones $b \times c$ cuesta abc multiplicaciones escalares con el algoritmo clásico.

Multiplicación de matrices

$$A_1 : 10 \times 30, \quad A_2 : 30 \times 5, \quad A_3 : 5 \times 60.$$

$$(A_1 A_2) A_3$$

Primero $A_1 A_2$:

$$10 \cdot 30 \cdot 5 = 1500.$$

El resultado tiene tamaño 10×5 . Luego:

$$10 \cdot 5 \cdot 60 = 3000.$$

Total: 4500.

$$A_1 (A_2 A_3)$$

Primero $A_2 A_3$:

$$30 \cdot 5 \cdot 60 = 9000.$$

El resultado tiene tamaño 30×60 . Luego:

$$10 \cdot 30 \cdot 60 = 18000.$$

Total: 27000.

Multiplicación de matrices

Algún algoritmo recursivo para resolver esto?

Multiplicación de matrices

Algún algoritmo recursivo para resolver esto? Toda parentización de $A_i \cdots A_j$ tiene una última multiplicación:

$$\underbrace{(A_i \cdots A_k)}_{d_{i-1} \times d_k} \underbrace{(A_{k+1} \cdots A_j)}_{d_k \times d_j}, \quad i \leq k < j.$$

Para un corte k fijo

- 1 Parentizamos la cadena izquierda.
- 2 Parentizamos la cadena derecha.
- 3 Hacemos recursión y sumamos el costo de la última recursión.

Costo de combinar

Las dimensiones de ambos resultados dependen solo de i, k, j , por lo que el último producto cuesta

$$d_{i-1} d_k d_j.$$

Multiplicación de matrices

Hagamos ahora la función recursiva.

Multiplicación de matrices

Hagamos ahora la función recursiva. Definimos $m(i, j)$ como el mínimo costo de multiplicar $A_i \cdots A_j$.

Algoritmo de backtracking?

$$m(i, j) = \begin{cases} 0, & i = j, \\ \min_{i \leq k < j} \{m(i, k) + m(k + 1, j) + d_{i-1}d_kd_j\}, & i < j. \end{cases}$$

- La respuesta buscada es $m(1, n)$.

Multiplicación de matrices

CADENA(i, j)

si $i = j$: **devolver** 0

$mejor \leftarrow +\infty$

para $k = i, \dots, j - 1$

$a \leftarrow \text{CADENA}(i, k)$

$b \leftarrow \text{CADENA}(k + 1, j)$

$c \leftarrow a + b + d_{i-1}d_kd_j$

$mejor \leftarrow \min\{mejor, c\}$

devolver $mejor$

Complejidad de backtracking

Cuánto pesa cada llamado?

Complejidad de backtracking

Cuánto pesa cada llamado? $O(n)$ Contemos ahora la cantidad de estados.

Complejidad de backtracking

Cuánto pesa cada llamado? $O(n)$ Contemos ahora la cantidad de estados. Sea $C(\ell)$ la cantidad de llamadas al resolver una cadena de ℓ matrices. Entonces

$$C(1) = 1, \quad C(\ell) = 1 + \sum_{s=1}^{\ell-1} (C(s) + C(\ell - s)) = 1 + 2 \sum_{s=1}^{\ell-1} C(s).$$

Complejidad de backtracking

Cuánto pesa cada llamado? $O(n)$ Contemos ahora la cantidad de estados. Sea $C(\ell)$ la cantidad de llamadas al resolver una cadena de ℓ matrices. Entonces

$$C(1) = 1, \quad C(\ell) = 1 + \sum_{s=1}^{\ell-1} (C(s) + C(\ell - s)) = 1 + 2 \sum_{s=1}^{\ell-1} C(s).$$

Restando la expresión correspondiente a $\ell - 1$ obtenemos, para $\ell \geq 2$,

$$C(\ell) - C(\ell - 1) = 2C(\ell - 1).$$

Por lo tanto,

$$C(\ell) = 3C(\ell - 1) = 3^{\ell-1}.$$

Complejidad final:

Complejidad de backtracking

Cuánto pesa cada llamado? $O(n)$ Contemos ahora la cantidad de estados. Sea $C(\ell)$ la cantidad de llamadas al resolver una cadena de ℓ matrices. Entonces

$$C(1) = 1, \quad C(\ell) = 1 + \sum_{s=1}^{\ell-1} (C(s) + C(\ell - s)) = 1 + 2 \sum_{s=1}^{\ell-1} C(s).$$

Restando la expresión correspondiente a $\ell - 1$ obtenemos, para $\ell \geq 2$,

$$C(\ell) - C(\ell - 1) = 2C(\ell - 1).$$

Por lo tanto,

$$C(\ell) = 3C(\ell - 1) = 3^{\ell-1}.$$

Complejidad final: $(n3^n)$.

La memoria permite calcular cada intervalo una sola vez

La función se llama con dos parámetros i, j . Podemos usar una matriz para memorizar los resultados intermedios.

CADENA-MEMO(i, j)

si $M[i, j] \neq \perp$: **devolver** $M[i, j]$

$mejor \leftarrow +\infty$

para $k = i, \dots, j - 1$

$a \leftarrow \text{CADENA-MEMO}(i, k)$

$b \leftarrow \text{CADENA-MEMO}(k + 1, j)$

$c \leftarrow a + b + d_{i-1}d_kd_j$

$mejor \leftarrow \min\{mejor, c\}$

$M[i, j] \leftarrow mejor$

devolver $M[i, j]$

También hay que inicializar la matriz en $\Theta(n^2)$.

Complejidad de PD

Cuál es la complejidad de PD?

Complejidad de PD

Cuál es la complejidad de PD? Hay n^2 estados distintos, donde cada uno se procesa en $O(n)$. Aparte, por la memorización cada estado se procesa una única vez (pensarlo como en el ej anterior).

Complejidad de PD

Cuál es la complejidad de PD? Hay n^2 estados distintos, donde cada uno se procesa en $O(n)$. Aparte, por la memorización cada estado se procesa una única vez (pensarlo como en el ej anterior).

Complejidad de PD

Sumando el costo de la inicialización, la cantidad de operaciones es

$$T(n) = \text{inicialización} + \text{recursión} = O(n^2) + O(n^2n) = O(n^3)$$

Algoritmo	Tiempo	Memoria adicional
Recursión sin memoria	$\Theta(3^n)$	$\mathcal{O}(n)$
Recursión con memoria	$\Theta(n^3)$	$\Theta(n^2)$

Qué algoritmo es mejor?

Complejidad de PD

Cuál es la complejidad de PD? Hay n^2 estados distintos, donde cada uno se procesa en $O(n)$. Aparte, por la memorización cada estado se procesa una única vez (pensarlo como en el ej anterior).

Complejidad de PD

Sumando el costo de la inicialización, la cantidad de operaciones es

$$T(n) = \text{inicialización} + \text{recursión} = O(n^2) + O(n^2n) = O(n^3)$$

Algoritmo	Tiempo	Memoria adicional
Recursión sin memoria	$\Theta(3^n)$	$\mathcal{O}(n)$
Recursión con memoria	$\Theta(n^3)$	$\Theta(n^2)$

Qué algoritmo es mejor? Ignorando el consumo de memoria, lo es el algoritmo de PD.

Complejidad de PD

Cuál es la complejidad de PD? Hay n^2 estados distintos, donde cada uno se procesa en $O(n)$. Aparte, por la memorización cada estado se procesa una única vez (pensarlo como en el ej anterior).

Complejidad de PD

Sumando el costo de la inicialización, la cantidad de operaciones es

$$T(n) = \text{inicialización} + \text{recursión} = O(n^2) + O(n^2n) = O(n^3)$$

Algoritmo	Tiempo	Memoria adicional
Recursión sin memoria	$\Theta(3^n)$	$\mathcal{O}(n)$
Recursión con memoria	$\Theta(n^3)$	$\Theta(n^2)$

Qué algoritmo es mejor? Ignorando el consumo de memoria, lo es el algoritmo de PD. **En la práctica van a ver una forma de bajar esta memoria a $O(n)$, dando un algoritmo estrictamente mejor.**

Conjuntto independiente

Problema: conjunto independiente máximo

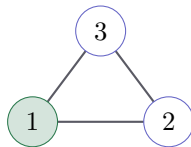
Dado un grafo simple no dirigido $G = (V, E)$, queremos encontrar un conjunto $I \subseteq V$ de máxima cardinalidad tal que

$$\text{para todo } u, v \in I, \quad \{u, v\} \notin E.$$

Al tamaño de este conjunto se lo suele denotar $\alpha(G)$.



En este camino, $I = \{1, 3, 5\}$ es óptimo: $\alpha(G) = 3$.



En un triángulo sólo podemos elegir un vértice: $\alpha(G) = 1$.

Conjunto independiente

Alguna solución recursiva?

Conjunto independiente

Alguna solución recursiva? Para $U \subseteq V$, definimos

$$F(U) = \alpha(G[U]), \quad N_G[v] = \{v\} \cup \{u : \{u, v\} \in E\}.$$

donde $G[U]$ conserva los vértices de U y las aristas entre ellos (es el inducido).

Idea para la recursión

Tenemos $F(\emptyset) = 0$. Si $U \neq \emptyset$, elegimos $v \in U$ y usamos

$$F(U) = \max \left\{ \underbrace{F(U \setminus \{v\})}_{\text{no elegir } v}, \quad \underbrace{1 + F(U \setminus N_G[v])}_{\text{elegir } v} \right\}.$$

Esta estrategia es análoga a la que vimos en la clase de demostraciones de backtracking.

Conjunto independiente

INDEPENDIENTE(U)

si $U = \emptyset$: **devolver** 0

$v \leftarrow \min U$

$a \leftarrow \text{INDEPENDIENTE}(U \setminus \{v\})$

$b \leftarrow 1 + \text{INDEPENDIENTE}(U \setminus N_G[v])$

devolver $\max\{a, b\}$

El árbol tiene profundidad a lo sumo...

Conjunto independiente

INDEPENDIENTE(U)

si $U = \emptyset$: **devolver** 0

$v \leftarrow \min U$

$a \leftarrow \text{INDEPENDIENTE}(U \setminus \{v\})$

$b \leftarrow 1 + \text{INDEPENDIENTE}(U \setminus N_G[v])$

devolver $\max\{a, b\}$

El árbol tiene profundidad a lo sumo... n y las llamadas recursivas son a lo sumo

Conjunto independiente

INDEPENDIENTE(U)

si $U = \emptyset$: **devolver** 0

$v \leftarrow \min U$

$a \leftarrow \text{INDEPENDIENTE}(U \setminus \{v\})$

$b \leftarrow 1 + \text{INDEPENDIENTE}(U \setminus N_G[v])$

devolver $\max\{a, b\}$

El árbol tiene profundidad a lo sumo... n y las llamadas recursivas son a lo sumo $O(2^{n+1})$. Cada llamada cuesta

Conjunto independiente

INDEPENDIENTE(U)

si $U = \emptyset$: **devolver** 0

$v \leftarrow \min U$

$a \leftarrow \text{INDEPENDIENTE}(U \setminus \{v\})$

$b \leftarrow 1 + \text{INDEPENDIENTE}(U \setminus N_G[v])$

devolver $\max\{a, b\}$

El árbol tiene profundidad a lo sumo... n y las llamadas recursivas son a lo sumo $O(2^{n+1})$. Cada llamada cuesta $O(n)$, por lo que la complejidad final es $O(n2^n)$.

Conjunto independiente con PD

- Podríamos aplicar memorización igual que antes. Cada llamado se corresponde con un subconjunto, así que necesitamos una estructura más compleja. Podemos usar máscaras, o sino un diccionario que permita indexar con conjuntos
- Esta estructura puede tener hasta 2^n entradas.

Observemos que **siempre** se puede aplicar memorización. A lo sumo hay que usar una estructura más compleja que un vector / matriz.

Complejidad de PD

- Si memorizamos, podemos garantizar que cada estado se visite una única vez.
- Pero ahora hay hasta 2^n estados distintos. Luego, no parece que haya una mejora tras garantizar visitarlos una única vez a cada uno.
- Peor aún, vamos a usar memoria exponencial en peor caso.

Algoritmo	Tiempo	Memoria adicional
Recursión sin memoria	$\Theta(n2^n)$	$\mathcal{O}(n)$
Recursión con memoria	$\Theta(n2^n)$	$\mathcal{O}(n2^n)$

Conjunto independientes

Un caso malo para PD

En el caso de conjunto independiente parecería que no se puede garantizar **superposición de subproblemas**. La fórmula recursiva que propusimos tiene hasta 2^n estados distintos, y por lo tanto memorizarlos no sirve demasiado.

- En el problema de la mochila y el de las varillas podíamos acotar la cantidad de estados distintos por $O(nW)$ y $O(n)$.
- Estas cotas salían de contar los posibles llamados: en general, los parámetros contenían la información que hay que mantener para seguir la recursión. **En el caso de independiente, tenemos que arrastrar toda la información.**
- Capaz podemos mejorar la recursión de conjunto independiente? En general, se cree que no...

Algunos principios generales sobre PD

Qué pasos seguimos para aplicar esta técnica?

- 1 **Plantear una recursión que resuelva el problema:** empezamos de un algoritmo de backtracking
- 2 **Elegir una estructura de memorización para los llamados:** hay que tener en cuenta el costo de inicialización, el de acceso / modificación y el consumo total de memoria

El algoritmo obtenido va a ser mejor o peor que antes de acuerdo a si podemos garantizar **superposición de subproblemas**. En general, esto se corresponde con probar que la **cantidad de estados distintos** es mucho más chica que la **cantidad de estados totales**. En el caso de las varillas, había hasta 2^n estados, pero solo n distintos.

Principios generales de PD

	Mochila	Varillas	Matrices	Independiente
Estado	(i, w)	ℓ	(i, j)	U
Estados totales	$\mathcal{O}(2^n)$	$\Theta(2^n)$	$\Theta(3^n)$	$\mathcal{O}(2^n)$
Estados distintos	$\mathcal{O}(nW)$	$\Theta(n)$	$\Theta(n^2)$	$\mathcal{O}(2^n)$
Trabajo por estado	$\mathcal{O}(1)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$
Tiempo sin memoria	$\mathcal{O}(2^n)$	$\Theta(n2^n)$	$\Theta(n3^n)$	$\mathcal{O}(n2^n)$
Tiempo con memorización	$\Theta(nW)$	$\Theta(n^2)$	$\Theta(n^3)$	$\mathcal{O}(n2^n)$

En general, contando las formas de parametrizar la función recursiva obtenemos una cota a la cantidad de llamados distintos.

Reconstrucción de la solución

Para la instancia que vimos del problema de la mochila teníamos que $F(3, 5) = 9$. Pero... Y si queremos encontrar una solución?

Vamos a ver que podemos adaptar el algoritmo que calcula el valor óptimo para que también devuelva la solución.

Esto sugiere el siguiente principio de diseño: nos olvidamos de la construcción, y solo pensamos en cómo calcular el valor óptimo. Luego, aplicamos estas técnicas de **reconstrucción** para recuperar la solución.

Reconstrucción

Si en (i, w) , con $i > 0$, y conocemos $M[i, w] = F(i, w)$... Como redescubrimos la decisión que hizo el backtracking?

Si $M[i, w] = M[i - 1, w]$

Reconstrucción

Si en (i, w) , con $i > 0$, y conocemos $M[i, w] = F(i, w)$... Como redescubrimos la decisión que hizo el backtracking?

Si $M[i, w] = M[i - 1, w]$

Podemos omitir el objeto i y pasar a $(i - 1, w)$ sin perder valor.

Si $M[i, w] > M[i - 1, w]$

Reconstrucción

Si en (i, w) , con $i > 0$, y conocemos $M[i, w] = F(i, w)$... Como redescubrimos la decisión que hizo el backtracking?

Si $M[i, w] = M[i - 1, w]$

Podemos omitir el objeto i y pasar a $(i - 1, w)$ sin perder valor.

Si $M[i, w] > M[i - 1, w]$

La alternativa óptima incluye al objeto i . Entonces

$$M[i, w] = b_i + M[i - 1, w - a_i].$$

Agregamos i a la solución y seguimos reconstruyendo a partir del estado $(i - 1, w - a_i)$.

En cada paso disminuye i : por lo que en n pasos terminamos. En qué complejidad se implementa esta estrategia?

Reconstrucción

Si en (i, w) , con $i > 0$, y conocemos $M[i, w] = F(i, w)$... Como redescubrimos la decisión que hizo el backtracking?

Si $M[i, w] = M[i - 1, w]$

Podemos omitir el objeto i y pasar a $(i - 1, w)$ sin perder valor.

Si $M[i, w] > M[i - 1, w]$

La alternativa óptima incluye al objeto i . Entonces

$$M[i, w] = b_i + M[i - 1, w - a_i].$$

Agregamos i a la solución y seguimos reconstruyendo a partir del estado $(i - 1, w - a_i)$.

En cada paso disminuye i : por lo que en n pasos terminamos. En qué complejidad se implementa esta estrategia? $O(n)$.

Ejemplo de reconstrucción

RECONSTRUIR-MOCHILA(M, n, W)

$L \leftarrow$ lista vacía; $i \leftarrow n$; $w \leftarrow W$

mientras $i > 0$

si $M[i, w] > M[i - 1, w]$
 agregar i al final de L

$w \leftarrow w - a_i$

$i \leftarrow i - 1$

devolver L

- Ejecutar después de MOCHILA-PD(n, W).
- Hace falta tener la memoria!

Ejemplo de reconstrucción

	capacidad w					
	0	1	2	3	4	5
$i = 3$	⊥	⊥	⊥	⊥	⊥	9
$i = 2$	⊥	⊥	⊥	5	⊥	8
$i = 1$	⊥	0	⊥	5	⊥	5
$i = 0$	0	0	0	0	0	0

Objeto	1	2	3
Peso	3	2	2
Valor	5	3	4

$W = 5$; óptimo 9.

En verde: estados del camino.

⊥: celda no calculada.

La figura muestra la tabla que deja MOCHILA-PD(3, 5), con la fila del fondo inicializada con el caso base.

Ejemplo de reconstrucción

Recordemos la recursión del problema de las varillas:

$$R(\ell) = \begin{cases} 0, & \ell = 0, \\ \max_{1 \leq k \leq \ell} \{p_k + R(\ell - k)\}, & \ell > 0. \end{cases}$$

Es decir, para cada ℓ , queremos buscar el k que cumple $R(\ell) = p_k + R(\ell - k)$.

Ejemplo de reconstrucción

Recordemos la recursión del problema de las varillas:

$$R(\ell) = \begin{cases} 0, & \ell = 0, \\ \max_{1 \leq k \leq \ell} \{p_k + R(\ell - k)\}, & \ell > 0. \end{cases}$$

Es decir, para cada ℓ , queremos buscar el k que cumple $R(\ell) = p_k + R(\ell - k)$.

Paso clave de la reconstrucción

```
q ← M[ℓ]
for k = 1 ... ℓ
  t ← pk + M[ℓ - k]
  si t = q
    Agregamos una varilla de tamaño k a la solución
    Seguimos la reconstrucción en ℓ - k.
```

Ejemplo de reconstrucción

En el problema de multiplicación de matrices, la memoria que cargamos tiene la siguiente estructura:

$$M[i, j] = \min_{i \leq k < j} \{M[i, k] + M[k + 1, j] + d_{i-1}d_kd_j\}.$$

Luego, empezando por el par $(1, n)$, queremos encontrar el índice que da la partición óptima, y luego hacer recursión para seguir buscando. Llamemos $S(i, j)$ a ese k , que se encuentra en $O(n)$.

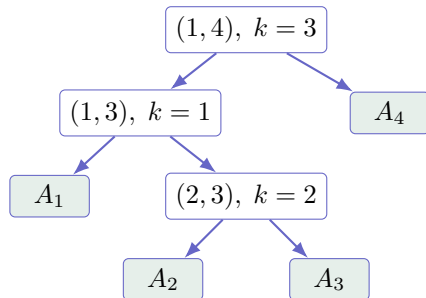
Ejemplo de reconstrucción

ESCRIBIR-PARÉNTESIS(S, i, j)

si $i = j$
 imprimir A_i

si no
 $k \leftarrow S[i, j]$
 imprimir "("
 ESCRIBIR-PARÉNTESIS(S, i, k)
 ESCRIBIR-PARÉNTESIS($S, k + 1, j$)
 imprimir ")"

Ejemplo de reconstrucción



Parentización óptima

$$((A_1(A_2A_3))A_4)$$

Conclusiones

- 1 En esta clase vimos varios ejemplos de uso de la técnica de PD.
- 2 Es una técnica que **se aplica sobre un algoritmo de backtracking**.
- 3 Siempre se puede aplicar, pero solo genera una ganancia si podemos garantizar **superposición de subproblemas**. Esta es una propiedad que los problemas pueden o no tener (comparar varillas versus conjunto independiente)
- 4 Finalmente, vimos que un algoritmo de decisión con programación dinámica puede convertirse fácilmente en uno de búsqueda usando **técnicas de reconstrucción**.